HomeFeast – Complete Project Analysis, Design System & Website Build Prompt

Oct 13, 2026 · @JEET

1. Project overview

HomeFeast is a responsive web platform where students, working professionals and elderly people find verified home cooks and subscribe to daily, weekly or monthly tiffin meals. Phase 1 is web only, built on a limited budget and a fixed timeline, and ends with four deliverables: a working web app, an admin dashboard, PRD plus technical documentation, and a deployment-ready build.

Item	What the brief says
Core idea	Digitize tiffin ordering, subscriptions and coordination between customer and home cook
Users	Customer, Home Cook / Tiffin Provider, Admin
In scope	Responsive web app, listings, subscriptions and orders, menu / price / availability display
Out of scope	Native apps, AI meal recommendations, cloud-kitchen integrations
Performance	Page load under 3 seconds
Security	Secure login, encrypted user data
Scalability	Multi-city onboarding
Suggested stack	React / Next.js, Tailwind or Bootstrap, Node + Express, MongoDB or PostgreSQL, AWS / Vercel / Netlify
Future phase	Online payments and wallet, mobile app, meal customization, calorie tracking, AI recommendations

Two small errors in the brief are worth fixing in your PRD: "Orders" is listed twice under Core Entities, and the text says "hygienichomemade" (missing space).

2. Problem analysis and solutions

People living away from home cannot easily find healthy, affordable, hygienic home food, and restaurant apps do not fix this. Each problem in the brief maps to a root cause, a platform solution and a feature you can demo.

#	Problem	Root cause	HomeFeast solution	Feature to build
1	High cost of meals	Restaurant pricing, delivery fees, commissions	Home cooks set their own low prices; no restaurant overhead	Transparent price on every plan, price filter, per-meal cost shown
2	Unhealthy or repetitive food	Restaurant menus are oil-heavy and fixed	Rotating weekly home-style menus by cooks	Weekly menu calendar, veg / non-veg tags, cuisine filter
3	No long-term subscription	Delivery apps only support one-off orders	Daily, weekly, monthly plans with pause and cancel	Subscription engine with start date, skip days, renewal
4	Limited visibility of local cooks	Home cooks have no online presence	Searchable, location-based cook directory	Nearby search by city / area, cook profile, ratings
5	Manual coordination by calls and WhatsApp	No system for requests, timings or tracking	Request, accept / reject, confirmation and history in one place	Cook dashboard, notifications, order and subscription tracking
6	Trust and hygiene concerns (implied by "verified")	Unknown cooks	Admin approval before listing, reviews and complaints	Cook verification workflow, rating system, dispute handling
7	Hard to scale beyond one area	Local, informal operations	City and area based data model	Multi-city schema, service-area per cook

Solving each problem step by step

1. **Cost:** show monthly price divided by meals so users compare tiffins fairly; offer weekly and monthly discounts.
2. **Variety:** let cooks publish a 7-day menu; show "today's menu" on the cook card.
3. **Subscription:** model a plan as cook + meal type + duration + days; the cook accepts or rejects it.
4. **Discovery:** index cooks by city, area / pincode and service radius; sort by distance, rating and price.
5. **Coordination:** status flow Pending → Accepted → Active → Paused / Cancelled / Completed, with email and in-app alerts at each change.
6. **Trust:** admin approves each cook after document check (ID and food safety / FSSAI details for India); only approved cooks appear in search.
7. **Scale:** every cook, menu and order carries a `cityId`, so a new city is data, not code.

3. Users, personas and user flow

Three roles use the platform, and each persona drives specific design choices.

Persona	Who	Main need	Design implication
Aarav, 21	Student in a hostel or PG	Cheap daily meals on a tight budget	Price filter first, monthly plan discounts, simple mobile UI
Neha, 28	Working professional	Healthy food on time, no cooking	Delivery-time slots, weekly menu view, pause / skip days
Mr. Shah, 68	Elderly person living alone	Light, simple, regular meals	Large text, easy re-order, phone-friendly contact for the cook
Meena, 42	Home cook	Steady extra income from her kitchen	Easy menu editing, clear request inbox, earnings view
Admin	Platform team	Quality control and trust	Approval queue, complaints, analytics

Customer user flow

1. Visit website
2. Register or log in
3. Search tiffin providers (nearby by city / area)

4. Apply filters and view cook profile and menu
5. Select meal plan (daily / weekly / monthly)
6. Place order or subscription
7. Cook accepts or rejects the request
8. User receives confirmation and tracks it in the dashboard

Cook onboarding flow

1. Register as cook and fill profile, kitchen photos, service area, timings
2. Upload ID / food-safety details
3. Admin approves or rejects with a reason
4. Cook adds menu items and plans
5. Cook starts receiving subscription requests

4. Requirements by role

Every requirement in the brief is listed below with an ID, priority (M = must, S = should, C = could) and a build note, so you can paste it straight into your PRD.

Customer

ID	Requirement	Priority	Build note
C-01	Register and log in (email + password, optional Google login)	M	JWT access + refresh token, email verification
C-02	Browse nearby tiffin providers	M	City / area / pincode search, optional geolocation
C-03	Filter by meal type (Veg / Non-Veg)	M	Query param `mealType`
C-04	Filter by price range	M	Min / max slider
C-05	Filter by plan (daily / weekly / monthly)	M	Plan type on each tiffin
C-06	Filter by cuisine	M	Cuisine managed by admin
C-07	View cook profile, menu, ratings	M	Public profile page with reviews

ID	Requirement	Priority	Build note
C-08	Subscribe or place a one-time order	M	Start date, days, address, notes
C-09	Track subscriptions and order history	M	Status timeline, filters
C-10	Rate and review a cook after delivery	S	One review per completed order
C-11	Pause, skip or cancel subscription	S	Cutoff time rule (e.g. before 8 PM previous day)
C-12	Raise a complaint	S	Linked to order, visible to admin

Home cook / tiffin provider

ID	Requirement	Priority	Build note
K-01	Dashboard to manage profile	M	Bio, photos, FSSAI / ID, languages
K-02	Add, edit, delete menu items	M	Dish name, meal type, price, availability, image
K-03	Set pricing and meal plans	M	Daily, weekly, monthly price per tiffin
K-04	Define service area and delivery timings	M	Areas / pincodes, lunch and dinner slots
K-05	Accept or reject subscription requests	M	Reason required on reject
K-06	View orders and earnings	M	Daily list, monthly total, simple chart
K-07	Mark orders delivered	S	Triggers review request
K-08	Holiday / leave mode	C	Auto-pauses availability

Admin

ID	Requirement	Priority	Build note
A-01	Manage users and cooks	M	Search, block, unblock, delete
A-02	Approve or reject cook registrations	M	Document preview, reason, email
A-03	Monitor orders and subscriptions	M	Filters by city, status, date
A-04	Manage categories and cuisines	M	CRUD
A-05	Handle complaints and disputes	M	Status: open, in review, resolved
A-06	KPI dashboard	S	Users, active subscriptions, conversion, retention
A-07	City management	S	Add new city without code change

Non-functional requirements

Area	Requirement	How to meet and test it
Performance	Page load under 3 seconds	Next.js SSR / ISR, image optimization, CDN, lazy loading; test with Lighthouse and WebPageTest (target Performance score 90+)
Security	Secure login, encrypted user data	See section 6
Usability	Simple, clean, mobile-first UI	Design for 360 px width first; touch targets of 44 px; WCAG 2.1 AA contrast
Scalability	Multi-city onboarding	City collection / table, indexes, stateless API, horizontal scaling
Availability	Reliable service	Health checks, uptime monitoring, daily backups
Maintainability	Clean documented code	ESLint, Prettier, Swagger docs, Git workflow

Area	Requirement	How to meet and test it
Accessibility	Usable by elderly users	Keyboard navigation, alt text, scalable fonts, light and dark mode

5. Complete technology stack

Use Next.js + Tailwind on the front, Node + Express on the back, and PostgreSQL as the database: this stays inside the brief's suggested stack and fits a subscription system with many relations (users, cooks, plans, orders). MongoDB also works if your mentor prefers it.

Layer	Technology	Why it is chosen
Frontend framework	Next.js (React) with TypeScript	SSR and ISR for fast loads and SEO of cook pages
Styling	Tailwind CSS + shadcn/ui, Lucide icons	Fast, consistent, mobile-first, easy dark mode
State and data fetching	TanStack Query, Zustand (or Context)	Caching and simple global state
Forms and validation	React Hook Form + Zod	Same schema can be reused on the server
Maps / location	Leaflet + OpenStreetMap (free) or Google Maps API	Nearby search without budget cost
Charts	Recharts	Cook earnings and admin KPIs
Backend	Node.js + Express.js (TypeScript)	As suggested in the brief
API style	REST, versioned `/api/v1`, documented with Swagger / OpenAPI	Orders, subscriptions, notifications
Database	PostgreSQL with Prisma ORM (alternative: MongoDB + Mongoose)	Relational integrity for subscriptions
Cache	Redis	Rate limiting, sessions, hot search results
File storage	Cloudinary or AWS S3	Dish and kitchen images, documents

Layer	Technology	Why it is chosen
Email and notifications	Nodemailer / Resend, in-app notifications, optional WhatsApp or SMS API later	Confirmation and status alerts
Scheduled jobs	node-cron or BullMQ	Daily order generation from subscriptions, renewals
Authentication	JWT + refresh tokens, bcrypt / Argon2, optional Google OAuth	Secure login
Testing	Jest, Supertest, React Testing Library, Playwright	Unit, API and end-to-end tests
Code quality	ESLint, Prettier, Husky	Clean consistent code
DevOps	Git + GitHub, GitHub Actions CI/CD, Docker	Repeatable builds
Deployment	Frontend on Vercel; backend on AWS (EC2 / Elastic Beanstalk), Render or Railway; DB on Neon, Supabase or AWS RDS	Deployment-ready build
Monitoring	Sentry, UptimeRobot, Lighthouse CI	Errors, uptime, 3-second target
Future	Razorpay / Stripe, React Native or PWA, calorie API	Payments, mobile, nutrition (phase 2)

Suggested folder structure

```
homefeast/
  client/        (Next.js app: app/, components/, hooks/, lib/, styles/)
  server/        (Express: routes/, controllers/, services/, middleware/,
models/, validators/, jobs/)
  prisma/        (schema.prisma, migrations, seed.ts)
  docs/          (PRD, API docs, ER diagram)
  docker-compose.yml
  .github/workflows/ci.yml
```

6. Security tech stack and practices

The brief asks for secure login and encrypted user data; the stack below covers that and the most common web attacks (OWASP Top 10) with tools that are free and easy to add in Express.

Area	Technology / practice	Protects against
Password storage	Argon2id or bcrypt (cost 12), never plain text	Database leaks
Authentication	Short-lived JWT access token (15 min) + refresh token in `HttpOnly`, `Secure`, `SameSite=Strict` cookie, with rotation	Token theft, XSS token reads
Authorization	Role-based access control (customer / cook / admin) middleware; check ownership on every record	Broken access control, IDOR
Encryption in transit	HTTPS everywhere (TLS 1.2+), HSTS	Man-in-the-middle
Encryption at rest	Encrypted database volume (AWS RDS / Atlas encryption); AES-256-GCM for sensitive fields such as phone, address, ID documents	Data exposure
Input validation	Zod / Joi on every request, parameterized queries through Prisma	SQL / NoSQL injection
Output safety	React escaping, DOMPurify for user text, Content Security Policy	XSS
HTTP headers	Helmet.js	Clickjacking, MIME sniffing
CSRF	SameSite cookies + CSRF token on cookie-based writes	CSRF
Rate limiting	express-rate-limit with Redis, stricter on login and OTP	Brute force, bots, DoS
CORS	Allow only your own frontend origin	Cross-site abuse
Account safety	Email verification, password reset with one-time expiring token, lockout after 5 failed logins, optional 2FA for admin	Account takeover

Area	Technology / practice	Protects against
File uploads	Check MIME type and size, rename files, store on S3 / Cloudinary, never execute	Malicious uploads
Secrets	`.env` files kept out of Git; AWS Secrets Manager or platform env vars	Leaked keys
Dependencies	npm audit, Dependabot, lockfile	Vulnerable packages
Logging and audit	Winston / Pino logs, admin action audit trail, Sentry; never log passwords or tokens	Undetected abuse
Privacy	Collect minimum data, user can delete account, privacy policy page	Compliance (India DPDP Act 2023)
Backups	Daily automated DB backups, tested restore	Data loss
Future payments	Use Razorpay / Stripe hosted checkout; never store card data	PCI scope

Security checklist before deployment

- [ ] All routes behind the correct role middleware
- [ ] Passwords hashed; no secrets in the repository
- [ ] HTTPS and HSTS enabled; Helmet configured
- [ ] Rate limiting active on auth routes
- [ ] Input validation on every endpoint
- [ ] Admin account protected with strong password and 2FA
- [ ] `npm audit` clean of high issues
- [ ] Backups and error monitoring running

7. Database design and API list

The brief lists seven core entities (Orders appears twice); the model below adds City, Cuisine, Plan and Complaint so multi-city support, filters and disputes work properly.

Entity	Key fields	Relations
User	id, name, email, phone, passwordHash, role, cityId, address, isVerified, isBlocked	Has many Orders, Subscriptions, Reviews
City	id, name, state, isActive	Has many Users and Cooks
Cook	id, userId, kitchenName, bio, photos, fssaiNo, serviceAreas, deliverySlots, status (pending / approved / rejected), ratingAvg	Belongs to User and City; has many Menus, Plans
Cuisine	id, name	Linked to Cook and Menu items
MenuItem	id, cookId, dishName, mealType (veg / non-veg), cuisineId, price, availability, deliveryTime, image	Belongs to Cook
Plan	id, cookId, name, type (daily / weekly / monthly), price, mealsPerDay, description	Belongs to Cook
Subscription	id, userId, cookId, planId, startDate, endDate, status, address, pausedDates	Belongs to User, Cook, Plan
Order	id, userId, cookId, subscriptionId (optional), items, total, deliveryDate, slot, status	Belongs to User, Cook
Review	id, orderId, userId, cookId, rating, comment	One per completed order
Complaint	id, orderId, userId, subject, details, status, adminNote	Handled by Admin
Notification	id, userId, type, message, isRead	Belongs to User

Main REST endpoints (`/api/v1`)

Area	Endpoints
Auth	POST `/auth/register` , `/auth/login` , `/auth/refresh` , `/auth/logout` , `/auth/forgot-password`

Area	Endpoints
Cooks (public)	GET `/cooks?city=&area=&mealType=&minPrice=&maxPrice=&plan=&cuisine=&sort=`, GET `/cooks/:id`, GET `/cooks/:id/menu`, GET `/cooks/:id/reviews`
Cook dashboard	GET / PUT `/cook/profile`, CRUD `/cook/menu`, CRUD `/cook/plans`, GET `/cook/requests`, PATCH `/cook/requests/:id` (accept / reject), GET `/cook/orders`, GET `/cook/earnings`
Subscriptions	POST `/subscriptions`, GET `/subscriptions/me`, PATCH `/subscriptions/:id` (pause / cancel)
Orders	POST `/orders`, GET `/orders/me`, PATCH `/orders/:id/status`
Reviews	POST `/reviews`, GET `/cooks/:id/reviews`
Complaints	POST `/complaints`, GET `/complaints/me`
Notifications	GET `/notifications`, PATCH `/notifications/:id/read`
Admin	GET `/admin/users`, PATCH `/admin/users/:id/block`, GET `/admin/cooks/pending`, PATCH `/admin/cooks/:id/approve`, CRUD `/admin/cuisines`, CRUD `/admin/cities`, GET `/admin/orders`, GET `/admin/complaints`, PATCH `/admin/complaints/:id`, GET `/admin/kpis`

Add a database index on `(cityId, status)` for cooks and on `(userId, status)` for subscriptions; this keeps search fast as cities are added.

8. Design system: light and dark theme

The look is warm and appetizing: saffron orange for action, leaf green for "fresh / veg / success", and cream or deep charcoal surfaces. Colors are stored as CSS variables, and the theme switches through a `data-theme` attribute (or the `dark` class in Tailwind), defaulting to the user's system setting and remembered after a manual toggle.

Token	Light mode	Dark mode	Used for
`--bg`	#FFFAF3	#14110F	Page background

Token	Light mode	Dark mode	Used for
`--surface`	#FFFFFF	#1E1A17	Cards, navbar, modals
`--surface-alt`	#FFF1DF	#2A2420	Sections, hover, table stripes
`--border`	#EADFCF	#3A322C	Dividers, input borders
`--text`	#2B2118	#F5EDE4	Main text
`--text-muted`	#6B5C4D	#B8A99A	Secondary text
`--primary`	#E8650A	#FF8A3D	Main buttons, links, highlights
`--primary-contrast`	#FFFFFF	#1A0F05	Text on primary buttons
`--secondary`	#2F7D4F	#4FBF7F	Veg badge, success, fresh accents
`--accent`	#F5B82E	#FFD05E	Ratings stars, offers
`--danger`	#C62F2F	#FF6B6B	Errors, reject, non-veg dot
`--info`	#2563EB	#6CA0FF	Info messages
`--shadow`	0 2px 12px rgba(60,40,20,.08)	0 2px 12px rgba(0,0,0,.5)	Card elevation

All text pairs above keep at least 4.5:1 contrast for body text in each mode (check final values with a contrast checker before submission).

```
:root {
  --bg: #FFFAF3;  --surface: #FFFFFF;  --surface-alt: #FFF1DF;  --border:
#EADFCF;
  --text: #2B2118;  --text-muted: #6B5C4D;
  --primary: #E8650A;  --primary-contrast: #FFFFFF;
  --secondary: #2F7D4F;  --accent: #F5B82E;  --danger: #C62F2F;  --info:
#2563EB;
}
:root[data-theme="dark"] {
  --bg: #14110F;  --surface: #1E1A17;  --surface-alt: #2A2420;  --border:
#3A322C;
  --text: #F5EDE4;  --text-muted: #B8A99A;
  --primary: #FF8A3D;  --primary-contrast: #1A0F05;
  --secondary: #4FBF7F;  --accent: #FFD05E;  --danger: #FF6B6B;  --info:
#6CA0FF;
}
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) { /* same values as dark block */ }
}
```

Typography, shape and components

Element	Choice
Headings	Poppins or Playfair Display (600 / 700)
Body	Inter or DM Sans, 16 px base, 1.6 line height
Corners	12 px cards, 999 px pills for badges and filters
Spacing	4 / 8 px scale
Icons	Lucide (leaf, utensils, clock, map-pin, star)
Veg / non-veg marker	Green square-with-dot for veg, red for non-veg (familiar to Indian users)
Key components	Navbar with theme toggle, hero search bar, filter chips and drawer, cook card, menu list, plan selector, subscription stepper, status badges, toasts, skeleton loaders, empty states
Motion	150–250 ms transitions; respect `prefers-reduced-motion`
Mobile	Bottom navigation (Home, Search, Orders, Profile); filters in a bottom sheet

9. Real-world features, KPIs, roadmap and risks

These touches make the project feel like a real product rather than a college demo, while staying inside the Phase 1 scope.

Real-world details to add

- Seed data for 2 cities, 10+ cooks, 40+ dishes, so the demo never looks empty
- Today's menu on each cook card, with a "Veg only" toggle
- Cutoff rule for skipping or pausing a meal (for example before 8 PM the previous day)
- Cook availability badge: "Accepting new subscribers" or "Full"
- Trial order or 1-day trial before a monthly plan
- Email confirmation on every accept, reject and cancel
- Cash on delivery or UPI at delivery in Phase 1 (online payments are a Phase 2 item in the brief)

- SEO-friendly cook pages with Open Graph previews, plus a PWA install option as a bridge to the future mobile app
- Admin analytics page showing the KPIs below

KPIs and how to calculate them

KPI	Formula / source	Where shown
Registered users	Count of users by role	Admin dashboard
Monthly active users	Users with a login or order in last 30 days	Admin dashboard
Active subscriptions	Subscriptions with status Active	Admin dashboard
Order conversion rate	Orders ÷ cook profile views × 100	Admin dashboard
Cook onboarding rate	Approved cooks ÷ cook sign-ups × 100	Admin dashboard
Customer retention rate	Customers renewing or reordering in month 2 ÷ customers in month 1 × 100	Admin dashboard

Suggested internship timeline (8 weeks)

1. **Week 1:** analysis, PRD, wireframes, design system, repo setup
2. **Week 2:** database schema, authentication, roles
3. **Week 3:** cook dashboard (profile, menu, plans, service area)
4. **Week 4:** customer search, filters, cook profile page
5. **Week 5:** subscription and order flow, accept / reject, notifications
6. **Week 6:** admin dashboard, approvals, complaints, KPIs
7. **Week 7:** reviews, dark mode polish, performance and security hardening, testing
8. **Week 8:** deployment, API and technical documentation, demo and final report

Risks and mitigations

Risk	Mitigation
Fixed timeline	Build Must items first; Should / Could items only if time remains

Risk	Mitigation
Limited budget	Free tiers: Vercel, Render, Neon / Supabase, Cloudinary, OpenStreetMap
Cooks submit unhygienic or fake profiles	Admin approval, document check, reviews, complaint flow
Slow pages with many images	Next.js image optimization, CDN, pagination
Too few cooks at launch	Seed data now; onboarding drive and referral link later
Scope creep (payments, AI)	Keep them in the Future Enhancements list only

10. Master prompt to build the website

Copy the prompt below into Claude, Cursor, v0 or another AI builder. For the best result, build in stages: first send the whole prompt and ask for the project setup and database, then ask for one module at a time (auth, cook dashboard, search, subscriptions, admin, polish).

```
You are a senior full-stack engineer and UI/UX designer. Build a production-quality, deployment-ready web application called "HomeFeast – Homemade Tiffin & Food Service Platform" for a software internship project.

## 1. PRODUCT CONTEXT
HomeFeast is a responsive web platform that connects users (students, working professionals, elderly people living away from home) with verified home cooks and tiffin providers offering fresh, hygienic, home-style meals on daily, weekly or monthly subscriptions. It replaces manual coordination through calls and WhatsApp and avoids expensive, repetitive restaurant food.
Phase 1 is web only. Out of scope: native mobile apps, AI meal recommendations, cloud-kitchen integrations, online payments (use Cash / UPI on delivery for now, but design the code so a payment gateway and wallet can be added later).

## 2. TECH STACK (use exactly this)
- Frontend: Next.js (App Router) + React + TypeScript, Tailwind CSS, shadcn/ui, Lucide icons, TanStack Query, React Hook Form + Zod, Recharts, Leaflet + OpenStreetMap
- Backend: Node.js + Express.js (TypeScript), REST API under /api/v1, Swagger/OpenAPI docs
- Database: PostgreSQL with Prisma ORM (include migrations and a seed script)
- Cache / rate limit: Redis (optional in dev, with in-memory fallback)
- Uploads: Cloudinary (or S3) for images and documents
```

```
- Email: Nodemailer (SMTP) with HTML templates
- Jobs: node-cron for daily order generation from active subscriptions
- Testing: Jest + Supertest (API), React Testing Library, one Playwright end-to-end test for the main flow
- DevOps: Docker + docker-compose, GitHub Actions CI, .env.example, ESLint + Prettier
- Deploy: frontend on Vercel, backend on Render/AWS, DB on Neon/Supabase/RDS. Provide deployment steps.

## 3. ROLES AND FEATURES
### Customer
- Register / login (email + password, Google OAuth optional), email verification, forgot/reset password
- Browse nearby tiffin providers by city / area / pincode (map and list views)
- Search and filter by: Veg / Non-Veg, price range, meal plan (daily / weekly / monthly), cuisine; sort by rating, price, distance
- View cook profile: kitchen photos, bio, FSSAI badge, service area, delivery slots, weekly menu, plans, ratings and reviews
- Subscribe or place a one-time order: choose plan, start date, meal slot (lunch / dinner), delivery address, notes
- Dashboard: active subscriptions, order history, status timeline, pause / skip / cancel (with cutoff time rule), rate and review after delivery, raise a complaint, notifications
### Home cook / tiffin provider
- Registration with documents; pending until admin approval
- Dashboard: manage profile, add / edit / delete menu items (dish name, meal type, price, availability, delivery time, image), set pricing and meal plans, define service areas and delivery timings, leave / holiday mode
- Accept or reject subscription requests (reason required on reject)
- View today's orders, mark delivered, view earnings (daily / monthly chart)
### Admin
- Manage users and cooks (search, block / unblock)
- Approve or reject cook registrations with document preview and reason
- Monitor orders and subscriptions with filters (city, status, date)
- Manage cuisines, categories and cities (add a new city without code changes)
- Handle complaints and disputes (open, in review, resolved, with notes)
- KPI dashboard: registered users, active subscriptions, order conversion rate, cook onboarding rate, monthly active users, customer retention rate

## 4. DATA MODEL (Prisma)
User, City, Cook, Cuisine, MenuItem, Plan, Subscription, Order, Review, Complaint, Notification. Every cook, menu item and order carries a cityId for multi-city scalability. Add indexes on (cityId, status) for cooks and (userId, status) for subscriptions. Subscription status flow: PENDING ->
```

```
ACCEPTED -> ACTIVE -> PAUSED / CANCELLED / COMPLETED (or REJECTED). Order status: PLACED -> CONFIRMED -> OUT_FOR_DELIVERY -> DELIVERED / CANCELLED. Cook status: PENDING -> APPROVED / REJECTED. Seed 2 cities (e.g. Rajkot and Ahmedabad), 12 cooks, 50 dishes, 5 cuisines, 3 customers, 1 admin, sample subscriptions, reviews and complaints.

## 5. DESIGN SYSTEM (light + dark mode)
Style: warm, clean, appetizing, mobile-first, modern food-tech look. Use CSS variables; support light and dark with a navbar toggle, default to system preference, remember the choice, no flash on load.
Light: bg #FFFAF3, surface #FFFFFF, surface-alt #FFF1DF, border #EADFCF, text #2B2118, text-muted #6B5C4D, primary #E8650A (text on it #FFFFFF), secondary/veg-green #2F7D4F, accent/star #F5B82E, danger #C62F2F, info #2563EB.
Dark: bg #14110F, surface #1E1A17, surface-alt #2A2420, border #3A322C, text #F5EDE4, text-muted #B8A99A, primary #FF8A3D (text on it #1A0F05), secondary #4FBF7F, accent #FFD05E, danger #FF6B6B, info #6CA0FF.
Typography: Poppins for headings, Inter for body, 16 px base. Rounded corners (12 px cards, pill badges). Veg = green square-dot marker, Non-Veg = red.
Components: sticky navbar with theme toggle, hero with search bar (city + meal type), filter chips and mobile bottom-sheet filters, cook cards with today's menu, plan selector, subscription stepper, status badges, toasts, skeleton loaders, empty states, bottom navigation on mobile. Subtle 150-250 ms transitions, respect prefers-reduced-motion. WCAG 2.1 AA contrast, visible focus states, keyboard accessible, large readable text options for elderly users.

## 6. PAGES
Public: Home (hero, how it works, featured cooks, plans, testimonials, FAQ, footer), Search / Listing, Cook Profile, Login, Register (customer or cook), Forgot/Reset password, About, Contact, Privacy, Terms, 404.
Customer: Dashboard, Subscriptions, Order history, Subscribe checkout, Reviews, Complaints, Notifications, Profile settings.
Cook: Dashboard overview, Profile, Menu manager, Plans and pricing, Service area and timings, Requests inbox, Orders, Earnings.
Admin: Overview KPIs, Users, Cook approvals, Orders and subscriptions, Cuisines and cities, Complaints.

## 7. SECURITY (mandatory)
- Passwords hashed with Argon2id or bcrypt (cost 12)
- JWT access token (15 min) + rotating refresh token in HttpOnly, Secure, SameSite cookie
- Role-based access control middleware and ownership checks on every record (prevent IDOR)
- HTTPS + HSTS, Helmet.js, strict CORS (own frontend origin only), CSRF protection for cookie-based writes
```

```
- Zod validation on every request body, query and param; parameterized queries only
- express-rate-limit (Redis store) with stricter limits on login, register and password reset; account lockout after 5 failed attempts
- Sanitize user-generated text (DOMPurify), Content Security Policy
- AES-256-GCM encryption for sensitive fields (phone, full address, ID documents); encrypted DB at rest in production
- Safe file uploads: MIME and size checks, renamed files, stored in Cloudinary/S3
- Secrets only in environment variables; .env never committed; npm audit and Dependabot in CI
- Audit log of admin actions; structured logging (Pino) that never logs secrets; Sentry for errors
- Admin 2FA (TOTP) optional but recommended; privacy policy and account deletion for DPDP Act 2023 alignment

## 8. NON-FUNCTIONAL TARGETS
- Page load under 3 seconds; Lighthouse Performance 90+ on mobile: use SSR/ISR, next/image, lazy loading, pagination, code splitting, caching headers, database indexes
- Mobile-first layouts tested at 360 px, 768 px, 1280 px
- Stateless API, ready for horizontal scaling and multi-city onboarding
- Clean architecture: routes -> controllers -> services -> repositories; centralized error handling; consistent JSON response format { success, data, message, errors }

## 9. DELIVERABLES
1. Full source code in a monorepo (client/, server/, prisma/, docs/) that runs with one docker-compose up
2. Admin dashboard
3. Swagger API documentation and a Postman collection
4. README with setup, environment variables, seed credentials, and deployment guide
5. PRD summary, ER diagram (Mermaid), user flow diagrams, and a short technical document in /docs
6. Automated tests and a GitHub Actions CI workflow
7. Deployment-ready production build

## 10. HOW TO WORK
Work in stages and confirm after each: (1) project structure, config, Docker, Prisma schema and seed; (2) auth and roles; (3) cook dashboard; (4) customer search and cook profile; (5) subscriptions and orders with notifications and the daily cron job; (6) admin dashboard and KPIs; (7) reviews, complaints,
```

```
dark mode polish, accessibility, performance and security hardening; (8)
tests, docs, deployment. Write complete working code (no placeholders or
TODOs), explain each file briefly, and state any assumptions. Start with
stage 1 now.
```

Shorter prompt for a front-end-only demo

If you only need a fast, good-looking prototype (for example in v0 or Lovable), use this:

```
Build a responsive, mobile-first React + Tailwind website for "HomeFeast", a
homemade tiffin subscription platform. Include a light and dark mode toggle
using a warm food theme (light: bg #FFFAF3, primary #E8650A, green #2F7D4F;
dark: bg #14110F, primary #FF8A3D, green #4FBF7F). Pages: Home with hero
search, cook listing with filters (veg / non-veg, price, daily / weekly /
monthly plan, cuisine), cook profile with weekly menu and reviews,
subscription flow, customer dashboard, cook dashboard (menu, plans, requests,
earnings), and admin dashboard (approve cooks, manage users, complaints,
KPIs). Use realistic mock data for Rajkot and Ahmedabad, skeleton loaders,
accessible components, and Lucide icons.
```